ASSIGNMENT-19.3

SHIVANI PAMBIDI

BATCH-41

2303A52172

TASK-1: Python to C++ Conversion

PROMPT:

Convert the given Python Student class into equivalent C++ code with the same structure and functionality. Make sure to include a proper constructor, correct data types, and use private and public access specifiers. Also, include all methods and a main() function, and ensure the C++ output matches the Python program.

Lab 9.py Lab exam.py sakila.db Lab 19.3.py

Lab 19.3.py > Student

```python
class Student:
    def __init__(self, name, age, marks):
        self.name = name
        self.age = age
        self.marks = marks

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Marks:", self.marks)

# Create object
s1 = Student("John", 20, 85)
s1.display()
```

CODE:

```python
# Define the Student class
class Student:
    # Constructor: Initializes the Student object
    # Takes name, age, and marks as parameters
    def __init__(self, name_param, age_param, marks_param):
        self.name = name_param     # Assign name
        self.age = age_param       # Assign age
        self.marks = marks_param   # Assign marks
    # Public method to display student information
    def display(self):
        print("Name:", self.name)    # Print name
        print("Age:", self.age)      # Print age
        print("Marks:", self.marks)  # Print marks
# Main program
# Create object of Student class
s = Student("Alice", 20, 95)

# Call display function
s.display()
```

OUTPUT:

```
•••  Name: Alice
     Age: 20
     Marks: 95
```

JUSTIFICATION:

This prompt ensures that the translated C++ code maintains the same structure and behavior as the original Python class. It highlights important differences like constructors, data types, and access specifiers, which are required in C++ but not explicit in Python. It also asks for a main() function and matching output to verify that the translation is correct and works properly.

TASK-2: Java to Python Function Conversion

PROMPT:

Convert a Java method that checks whether a number is prime into an equivalent Python function. Make sure to follow Python syntax and keep the same logic and behavior. Also, test the function with different inputs to verify correct output.

```java
public class PrimeCheck {

    public static boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[] testNumbers = {0, 1, 2, 3, 4, 5, 10, 17, 19, 20};

        for (int num : testNumbers) {
            if (isPrime(num)) {
                System.out.println(num + " -> Prime");
            } else {
                System.out.println(num + " -> Not Prime");
            }
        }
    }
}
```

CODE:

Lab 19.3.py > ...

```python

import math

def isPrime(n):
    if n <= 1:
        return False
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

testNumbers = [11, 12, 13, 14, 15, 16, 23, 29, 30]

for num in testNumbers:
    if isPrime(num):
        print(f"{num} -> Prime")
    else:
        print(f"{num} -> Not Prime")

```

OUTPUT:

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/Lab 19.3.py"
16 -> Not Prime
23 -> Prime
29 -> Prime
30 -> Not Prime
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding>
```

JUSTIFICATION:

This prompt ensures the Python function keeps the same logic as the Java method while adapting to Python's simpler syntax. It removes Java-specific elements like data types and class structure and uses Python's indentation and dynamic typing. Testing with multiple inputs helps confirm that the converted function works correctly and produces the same results as the Java version.

TASK-3: Pseudocode to Python Implementation

PROMPT:

Convert the given pseudocode for sorting numbers (bubble sort) into a working Python program. Make sure to follow Python syntax and keep the same step-bystep logic. Also, test the program with sample input lists to verify correct sorting.

PSEUDO CODE:

START

FOR i from 0 to n-1

  FOR j from 0 to n-i-2

    IF arr[j] > arr[j+1]

      SWAP arr[j] and arr[j+1]

END

CODE:

Lab 19.3.py > ...

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # swap
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Test lists (changed numbers)
test_list1 = [45, 12, 78, 34, 23]
test_list2 = [99, 11, 56, 2, 67]

# Sorting the lists
sorted_list1 = bubble_sort(test_list1)
sorted_list2 = bubble_sort(test_list2)

# Display the results
print("Sorted List 1:", sorted_list1)
print("Sorted List 2:", sorted_list2)
```

OUTPUT:

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Pro
grams/Python/Python313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/Lab
 19.3.py"
Sorted List 1: [12, 23, 34, 45, 78]
Sorted List 2: [2, 11, 56, 67, 99]
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding>
```

JUSTIFICATION:

This prompt ensures that the pseudocode logic is accurately converted into Python while keeping the same step-by-step sorting process. It helps in understanding how loops, conditions, and swapping in pseudocode are implemented using Python syntax. By testing with different input lists, we can verify that the algorithm works correctly in all cases and produces properly sorted results. It also improves problem-solving skills by showing how abstract logic (pseudocode) is turned into a real, executable program.

TASK-4: SQL to Pandas Query

PROMPT:

Convert the given SQL query into an equivalent Pandas DataFrame operation in Python while keeping the same logic for filtering and selecting columns. Make

sure to use proper Pandas syntax and clearly show how the data is stored in a DataFrame. Also, test the code with sample data to ensure the output matches the SQL query result.

SELECT name, salary

FROM employees WHERE

salary > 50000; CODE:

```python
import pandas as pd
# Sample DataFrame
data = {
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'salary': [60000, 45000, 70000, 30000]
}
df = pd.DataFrame(data)
# Equivalent Pandas query
result = df[df['salary'] > 50000][['name', 'salary']]
# Output
print(result)
```

OUTPUT:

```
      name  salary
0    Alice   60000
2  Charlie   70000
```

JUSTIFICATION:

This prompt ensures that the SQL query logic is correctly translated into Pandas operations while keeping the same filtering and column selection. It helps in understanding how database queries can be performed using Python on DataFrames. Testing with a sample dataset verifies that the output matches the SQL query results. This task also improves understanding of data handling in Python and highlights the similarity between SQL and Pandas operations.

TASK-5: Real-Time Application: Algorithm Translation Across

PROMPT:

Convert a Python searching algorithm (linear search) into an equivalent C program while keeping the same logic and functionality. Make sure to handle data types, loops, and input/output properly. Also, test both versions to ensure they produce the same result.

```python
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

arr = [10, 20, 30, 40, 50]
key = 30

result = linear_search(arr, key)

if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

CODE:

```python
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

# Sample input
arr = [10, 20, 30, 40, 50]
key = 30

# Function call
result = linear_search(arr, key)

# Output
if result != -1:
    print("Element found at index:", result)
else:
    print("Element not found")
```

OUTPUT:

```
Element found at index: 2
```

JUSTIFICATION:

This prompt ensures that the algorithm is translated correctly between two different programming languages while maintaining the same logic. It highlights key differences such as Python's simple syntax versus C's need for explicit data types and memory handling. Testing both versions helps verify correctness and consistency of output. This task also demonstrates real-world challenges like syntax differences, lack of built-in functions in C, and manual handling of arrays.